

Technical Disclosure Commons

Defensive Publications Series

16 Aug 2024

EFFICIENT NETWORK AUTOMATION THROUGH ENHANCED RETRIEVAL AUGMENTED GENERATION (RAG) ARCHITECTURE

Koushik Padavalli Gowrish

Supratim Chakraborty

Somaraju Kaki

Nithin Chitta

Mahesh Chittur Kalyanakrishnan

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

Gowrish, Koushik Padavalli; Chakraborty, Supratim; Kaki, Somaraju; Chitta, Nithin; and Kalyanakrishnan, Mahesh Chittur, "EFFICIENT NETWORK AUTOMATION THROUGH ENHANCED RETRIEVAL AUGMENTED GENERATION (RAG) ARCHITECTURE", Technical Disclosure Commons, (August 16, 2024)
https://www.tdcommons.org/dpubs_series/7284



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

EFFICIENT NETWORK AUTOMATION THROUGH ENHANCED RETRIEVAL AUGEMENTED GENERATION (RAG) ARCHITECTURE

AUTHORS:

Koushik Padavalli Gowrish
Supratim Chakraborty
Somaraju Kaki
Nithin Chitta
Mahesh Chittur Kalyanakrishnan

ABSTRACT

Proposed herein is an enhanced retrieval augmented generation (E-RAG) architecture for use with network automation applications. The enhanced RAG architecture aims to increase RAG performance by reducing the hallucinations of a Large Language Model (LLM) and by decreasing the context length of messages sent to the LLM.

DETAILED DESCRIPTION

Conversational chatbots for network automation are on the rise. For example, many chatbots are trained on internal documents related to network infrastructure, configuration guides etc. Currently, determining the live status of a system in a network through natural language queries (e.g., state of the network, etc.) often relies on state-of-the-art LLM models.

In such scenarios, the LLM needs to identify the exact Command Line Interface (CLI) command that it must call/execute via a node of the network in order to obtain the status of the network/node. However, LLMs have limited context length and are prone to hallucinations. Generally, limited context means that the number of CLI commands that are supported by an LLM to obtain network/node status is reduced.

Retrieval augmented generation (RAG) architectures were introduced to reduce hallucinations and provide in-context response; Though they reduce context limit problem by performing similarity searches on a query and vector databases to some extent, they typically don't output an exact CLI command and are still prone to hallucinations.

Function calling features in addition to RAG architecture can reduce hallucination problem as the output of an LLM after using a function calling feature can be mapped to a

CLI. However, the performance of a functional calling RAG architecture may not be accurate and can be erroneous when the output of a similarity search has two or more similar meaning CLI descriptions. A conventional RAG architecture is shown below in Figure 1.

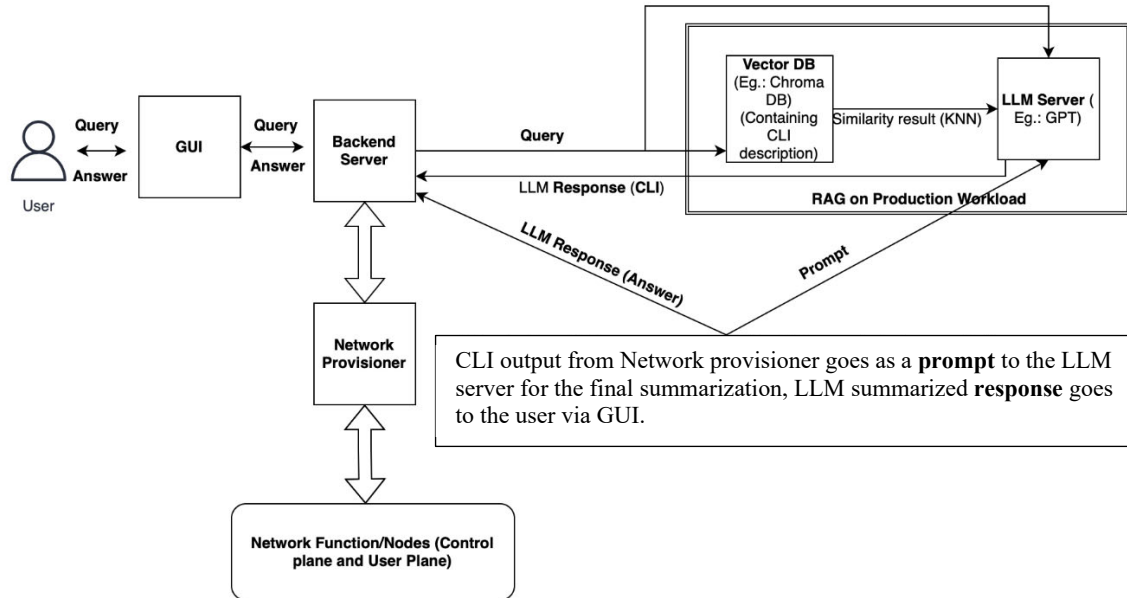


Figure 1: Conventional RAG Architecture

To address such issues/limitations of conventional LLM and RAG architectures, an enhanced RAG architecture is proposed herein that provides for adding a transformer network to the existing RAG architecture with function calling. Figure 2, below, illustrates example details and building blocks of the enhanced RAG architecture proposed herein.

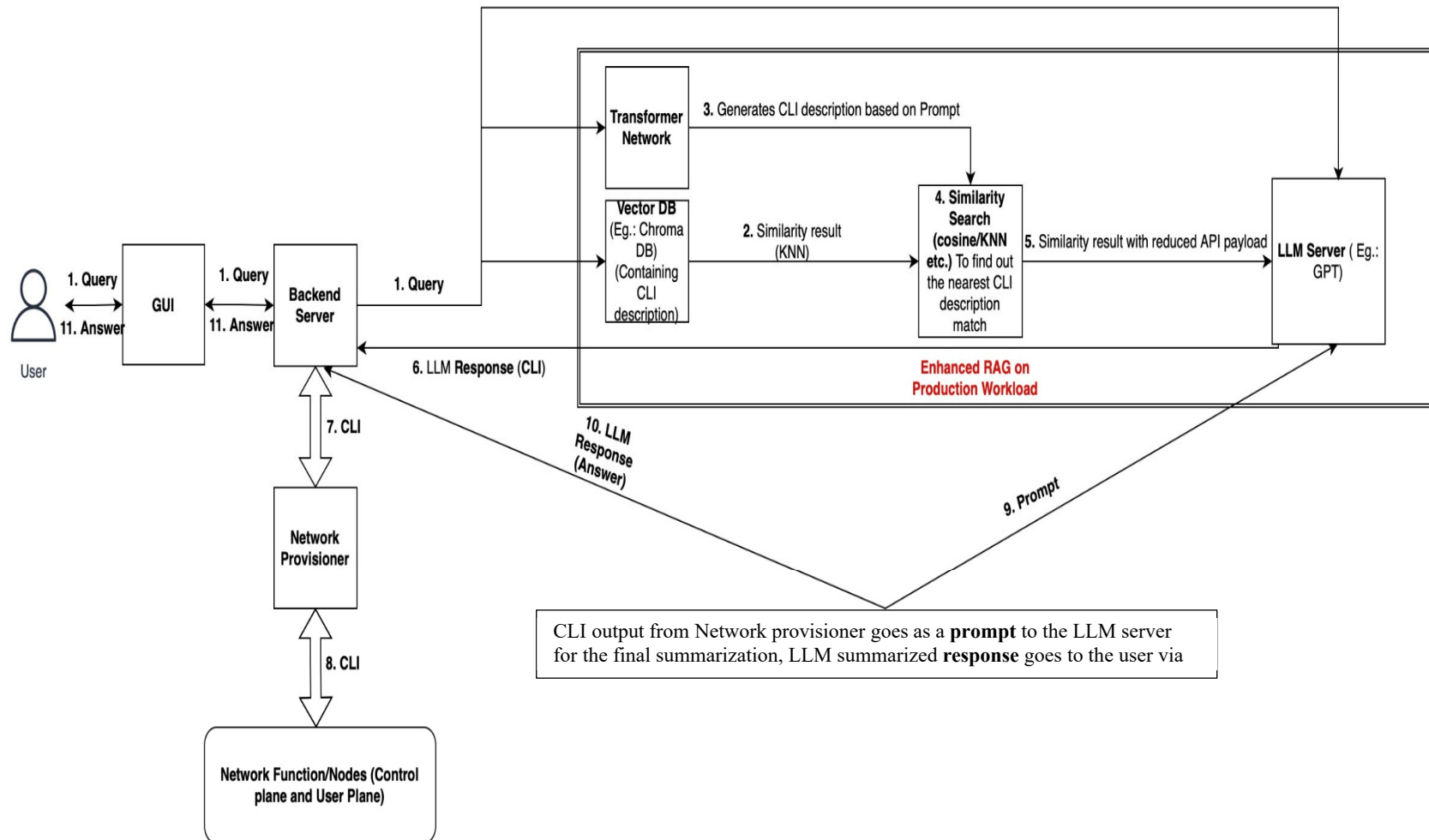


Figure 2: Enhanced RAG Architecture

Consider various operations that can be realized through the enhanced RAG architecture, as proposed herein, that may increase RAG performance by reducing the hallucinations of an LLM and may also decrease the context length of messages sent to the LLM.

As shown in Figure 2, consider at (1) that a user can ask a query through a graphical user interface (GUI) that is sent to a backend server. As shown at (2), the query can be compared against a vector database that contains CLI and functional descriptions corresponding to different potential CLIs that can be called. A similarity search is performed against the query and vector database that can result in a first level selection of CLIs and their corresponding descriptions.

The query is also forwarded to a transformer network, as shown at (3), that can be trained in such a way that it can output a CLI description based on a given input query.

A second level of selection can be performed, as shown at (4), via a similarity search block (cosine search/K-Nearest Neighbor (KNN)) at which the CLI description output of the transformer network can be compared against the output of the first similarity search (and corresponding CLI descriptions).

This second level of filtering may be useful in scenarios in which there are two or more similar CLIs having the same/approximately the same similarity scores. In a traditional RAG approach, there is a chance of an LLM choosing an erroneous CLI since the CLIs input to the LLM are similar. Thus, adding this second layer of filtering of CLIs not only improves the performance of the enhanced RAG architecture of this proposal, but also controls and reduces the input message context length to the LLM.

Moving to (5), the similarity search result with a reduced application programming interface (API) payload is sent to the LLM, where the final level of selection of a corresponding CLI is performed. As shown at (6), the LLM outputs the CLI command to be executed (via the node/network for which status is requested) and is sent to the backend server.

As shown at (7), the backend server communicates with the network provisioner to execute (fire) the CLI in the appropriate network function and the network provisioner securely executes (8) the CLI and provides the output to the backend server. The backend server then sends the prompt (CLI + CLI output + input query) to the LLM, as shown at

(9), The LLM answers the user query (10) based on the system prompt and the answer can be sent to the backend server. The final answer can then be displayed in the GUI that is sent to the user, as shown at (11).

Accordingly, the enhanced RAG architecture, as proposed herein, may increase RAG performance by reducing the hallucinations of an LLM and may also decrease the context length of messages sent to the LLM utilizing various techniques and operations as illustrated and discussed above with reference to Figure 2.